



Listas y Mapas

- ¿Qué son? Definición y conceptos
- Demostración con código
- Uso y aplicaciones prácticas
- Comparación entre Listas y Mapas

Agenda de la presentación

01

¿Qué son las Listas y los Mapas?

Definición, características y tipos

02

Demostración con código Dart

Ejemplos prácticos y sintaxis

03

Usos y aplicaciones prácticas

Casos reales en Flutter

04

Comparación: Listas vs Mapas

Ventajas, desventajas y cuándo usar cada uno

¿Qué es una Lista en Dart?

– Definición

Una Lista (List) es una colección ordenada de elementos del mismo tipo. En Dart, equivale a un array dinámico que puede crecer o reducirse.

Características clave

- Ordenada: mantiene el orden de inserción
- Indexada: acceso por posición (0, 1, 2...)
- Acepta duplicados
- Puede ser fija (List.filled) o dinámica
- Tipado: List<int>, List<String>, etc.

Tipos de Listas

Crecible

```
var l = [];  
l.add(1);
```

Fija

```
List.filled(3, 0)
```

Solo lectura

```
List.unmodifiable()
```

¿Qué es un Mapa en Dart?

Definición

Un Mapa (Map) es una colección de pares clave-valor. Cada clave es única y se usa para acceder a su valor asociado. Equivale a un diccionario.

Características clave

- No ordenado por defecto
- Claves únicas (no se repiten)
- Valores pueden repetirse
- Acceso $O(1)$ por clave (HashMap)
- Tipado: `Map<String, int>`, etc.

Analogía: clave → valor



Tipos de Mapas en Dart

HashMap

Acceso rápido $O(1)$;
orden no garantizado

LinkedHashMap

Mantiene orden de
inserción (default)

SplayTreeMap

Ordenado por clave;
operaciones $O(\log n)$

Demostración – Listas en Dart

Código

```
// Declaración de listas
List<String> frutas = ['Mango', 'Pera', 'Uva'];

// Lista dinámica
var numeros = <int>[];
numeros.add(10);
numeros.addAll([20, 30, 40]);

// Acceder elementos
print(frutas[0]);    // Mango
print(frutas.length); // 3

// Iterar con for-each
frutas.forEach((f) => print(f));

// Métodos útiles
frutas.remove('Pera');
frutas.sort();
var rev = frutas.reversed.toList();
```

Métodos más usados

<code>.add(e)</code>	Agrega elemento al final
<code>.remove(e)</code>	Elimina primera coincidencia
<code>.contains(e)</code>	Verifica si existe
<code>.indexOf(e)</code>	Retorna índice
<code>.sort()</code>	Ordena la lista
<code>.map(fn)</code>	Transforma elementos
<code>.where(fn)</code>	Filtra elementos
<code>.sublist(i,j)</code>	Sublista de i a j
<code>.isEmpty</code>	Verifica si está vacía

Demostración – Mapas en Dart

– Código

```
// Declaración de un Map
Map<String, dynamic> usuario = {
  'nombre': 'Carlos',
  'edad': 22,
  'activo': true,
};

// Acceder y modificar
print(usuario['nombre']); // Carlos
usuario['edad'] = 23;

// Agregar y eliminar entradas
usuario['email'] = 'c@mail.com';
usuario.remove('activo');

// Iterar claves y valores
usuario.forEach((k, v) {
  print('$k: $v');
});
```

Métodos más usados

<code>.containsKey(k)</code>	Verifica si existe la clave
<code>.containsValue(v)</code>	Verifica si existe el valor
<code>.remove(k)</code>	Elimina entrada por clave
<code>.keys</code>	Iterable de claves
<code>.values</code>	Iterable de valores
<code>.entries</code>	Iterable de MapEntry
<code>.length</code>	Número de entradas
<code>.isEmpty</code>	Verifica si está vacío
<code>.addAll(map)</code>	Fusiona otro mapa

Uso de Listas en Flutter

– Aplicaciones

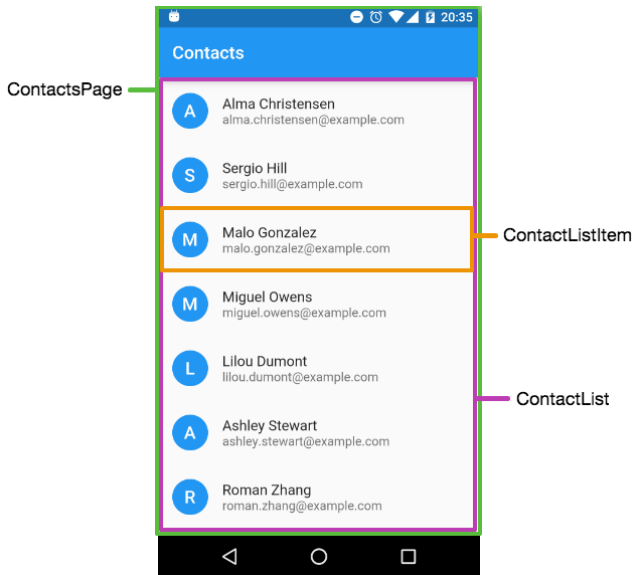
Aplicaciones de las listas

- Mostrar productos y contactos.
- Chats y redes sociales.
- Menús dinámicos .
- Datos obtenidos de APIs.

Widgets relacionados

ListView

ListView.builder



Código de ejemplo

```
List<String> productos = [
  "Laptop",
  "Mouse",
  "Teclado"
];
```

```
ListView.builder(
  itemCount: productos.length,
  itemBuilder: (context, index) {
    return Text(productos[index]);
  },
)
```

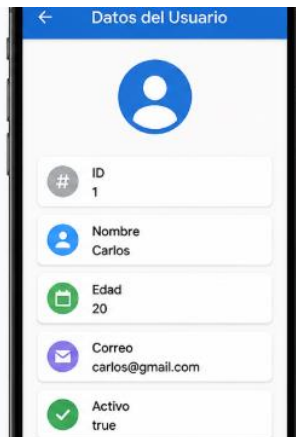
Uso de Mapas en Flutter

– Aplicaciones

Aplicaciones de los mapas

- Manejo de JSON y APIs
- Datos de usuarios
- Configuración de aplicaciones
- Traducciones e idiomas

Clave (Key)	Valor (Value)
id	1
nombre	Carlos
edad	20
correo	carlos@gmail.com
activo	true



Código de ejemplo

```
Map<String, dynamic> usuario = {  
  "nombre": "Carlos",  
  "edad": 20  
};
```

```
Text(usuario["nombre"])
```


Aplicación real de ambas

– Aplicaciones

Caso real en Flutter

- APIs REST
- Datos dinámicos
- Productos y usuarios
- Integración con ListView

Código de ejemplo

```
List<Map<String, dynamic>>
productos = [
  {
    "nombre": "Laptop",
    "precio": 800
  },
  {
    "nombre": "Mouse",
    "precio": 25
  }
];
```

Comparación: Listas vs Mapas

Comparación		
	LISTA	MAPA
Acceso	Por índice numérico	Por clave única
Orden	Siempre ordenada	No garantizado (HashMap)
Duplicados	Permite duplicados	Claves únicas; valores dup. ok
Búsqueda	$O(n)$ lineal	$O(1)$ por clave
Memoria	Menor overhead	Mayor overhead
Iteración	Simple (for, forEach)	Por clave, valor o entry
JSON / API	Listas JSON []	Objetos JSON {}

Complejidad Big O

¿Qué es Big O?

- Notación que mide la eficiencia de una operación en función del número de elementos.
- No mide tiempo en segundos, sino pasos o comparaciones.
- Y responde la pregunta: ¿Cómo crece el costo cuando crecen los datos?

Notaciones principales

- **$O(1)$** — Tiempo constante: el costo no depende del tamaño de la colección
- **$O(n)$** — Tiempo lineal: el costo crece proporcionalmente al número de elementos
- **$O(n^2)$** — Tiempo cuadrático: el costo crece de forma exponencial (loops anidados)

¿Cuándo usar cada uno – Regla de Decisión?

Guía práctica

Usa una LISTA cuando...

- El orden importa (ranking, pasos)
- Accedes elementos por posición
- Tienes colección de ítems similares
- Necesitas iterar todos los elementos
- Construyes un ListView o GridView
- Almacenas resultados de una API []
- Implementas una pila o cola

Usa un MAPA cuando...

- Necesitas buscar por clave ($O(1)$)
- Los datos tienen etiquetas (nombre, edad...)
- Parseas JSON de un backend
- Configuras rutas o localización
- Necesitas asociar dos valores
- Cuentas frecuencias (word count)
- Evitas múltiples variables sueltas